

II. Les bases & bonnes pratiques HTML5

HTML5 SÉMANTIQUE EN UNE PHRASE

HTML5 n'est pas qu'une liste de balises : c'est une grammaire qui donne du **SENS** à chaque morceau de contenu, pour les navigateurs, les moteurs de recherche et les lecteurs d'écran.

Sommaire

- **Introduction**
 - 1. Objectifs du chapitre
 - 2. En pratique — situation professionnelle
 - 3. Quand l'utiliser, quand l'éviter
- **A. Structure d'une page HTML5**
 - 1. Squelette minimal
 - 2. Évolution de HTML
- **B. Balises sémantiques**
 - 1. Balises de structure
 - 2. `section` vs `article`
- **C. Hiérarchie des titres et mise en valeur**
 - 1. Hiérarchie de titres
 - 2. Mise en valeur sémantique vs présentationnelle
- **D. Métadonnées et SEO**
 - 1. Balises `meta` essentielles
 - 2. Open Graph (partage social)
- **E. Accessibilité de base**
 - 1. Alt-text sur les images
 - 2. ARIA et labels
 - 3. Skip links
- **F. Conventions de nommage et organisation**
 - 1. Méthodologie BEM
 - 2. Arborescence de fichiers recommandée
 - 3. Validation
- **Conclusion**
 - 1. À retenir
 - 2. Pour aller plus loin

Introduction

1. Objectifs du chapitre

À l'issue de ce chapitre, vous serez capable de :

- **Identifier** les balises sémantiques HTML5 (`<header>` , `<nav>` , `<main>` , `<article>` , `<section>` , `<footer>`).
- **Distinguer** les balises présentationnelles (`<b>` , `<i>`) des balises sémantiques (`<strong>` , `<em>`).
- **Appliquer** une hiérarchie de titres cohérente (h1 → h6 sans saut).
- **Écrire** les métadonnées essentielles (`charset` , `viewport` , `description`) pour le SEO et la compatibilité mobile.
- **Évaluer** une page HTML selon les critères d'accessibilité de base (alt-text, hiérarchie, skip links).

2. En pratique — situation professionnelle

Votre client reçoit un rapport Lighthouse avec un score accessibilité de 62. Les erreurs : *heading levels skipped*, *buttons without discernible name*, *images without alt attribute*. Ce chapitre vous donne les réflexes HTML qui évitent ces erreurs dès l'écriture, avant que Lighthouse ne les remonte.

3. Quand l'utiliser, quand l'éviter

✅ Utiliser pour	❌ Éviter quand
Toute nouvelle page HTML (squelette + sémantique systématique)	Sortie d'un export généré (PDF, Markdown) qui n'a pas besoin de SEO
Préparer un audit Lighthouse / WAVE / RGAA réussi	Maquette jetable de 5 minutes sans déploiement
Travail en équipe (BEM stabilise les classes)	Prototype solo où la convention de nommage ralentit

A. Structure d'une page HTML5

1. Squelette minimal

Toute page HTML5 valide commence par le même squelette :

```
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Titre de la page</title>
  </head>
  <body>
    <!-- contenu visible -->
  </body>
</html>
```

- `<!DOCTYPE html>` — déclaration HTML5, obligatoire en première ligne.
- `lang="fr"` — indique la langue principale aux lecteurs d'écran et moteurs de recherche.
- `<meta charset="UTF-8">` — encodage universel, doit apparaître **avant tout contenu**.
- `<meta name="viewport">` — indispensable pour le responsive mobile.
- `<title>` — texte de l'onglet et du résultat Google.

2. Évolution de HTML

Année	Version	Apport clé
1991	HTML 1	Premières balises (titre, lien, paragraphe)
1997	HTML 4	Tables, formulaires, scripts
2000	XHTML	Syntaxe stricte type XML
2014	HTML5	Balises sémantiques, multimédia natif, APIs

HTML5 a introduit plus de **100 balises sémantiques** pour résoudre le problème de la *div soup* (usage excessif de `<div>` sans signification).

B. Balises sémantiques

1. Balises de structure

```
<body>
  <header>...</header>      <!-- en-tête : logo, titre, nav principale -->
  <nav>...</nav>            <!-- menu principal -->
  <main>                     <!-- contenu principal, UN SEUL par page -->
    <article>...</article>
    <section>...</section>
    <aside>...</aside>      <!-- contenu connexe (barre latérale) -->
  </main>
  <footer>...</footer>      <!-- copyright, liens sociaux -->
</body>
```



Règles à retenir :

- `<main>` : **un seul par page**. Ne doit pas contenir de `<header>`, `<footer>` ni `<nav>` principale.
- `<header>` et `<footer>` : **multiples autorisés** (un global de page, un par article par exemple).
- `<nav>` : réservé aux **menus principaux**, pas aux simples listes de liens.

2. section VS article

Distinction clé

- `<article>` = contenu **autonome**, qui aurait du sens publié seul (article de blog, post social, fiche produit).
- `<section>` = **regroupement thématique** de contenu lié, qui perd son sens si extrait.

Un `<article>` peut contenir plusieurs `<section>`. Une `<section>` peut contenir plusieurs `<article>`.

Exemple concret :

```
<article>
  <h2>Comment utiliser Docker</h2>
  <section>
    <h3>Installation</h3>
    <p>...</p>
  </section>
  <section>
    <h3>Premiers conteneurs</h3>
    <p>...</p>
  </section>
</article>
```

C. Hiérarchie des titres et mise en valeur

1. Hiérarchie de titres

```
<h1>Titre principal (un seul par page)</h1>
<h2>Sous-titre majeur</h2>
<h3>Sous-section</h3>
←!— jusqu'à h6 —→
```

⚠ Ne jamais sauter de niveaux

- ✗ h1 → h3 → h6
- ✓ h1 → h2 → h3

Les lecteurs d'écran utilisent la hiérarchie pour permettre à l'utilisateur de *sauter* d'une section à l'autre. Un saut casse cette navigation.

2. Mise en valeur sémantique vs présentationnelle

Sémantique (préféré)	Présentationnel (à éviter)	Usage
<code></code>	<code></code>	Importance forte du texte
<code></code>	<code><i></code>	Emphase, accent
<code><mark></code>	(aucun)	Surlignage contextuel
<code></code> / <code><ins></code>	<code><s></code> / ~	Suppression / insertion de contenu

La règle : si la mise en forme a un **sens** (c'est important, c'est accentué), utiliser une balise sémantique. Si c'est purement visuel (italique typographique sans intention d'emphase), utiliser `<i>` ou CSS.

D. Métadonnées et SEO

1. Balises meta essentielles

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Description concise de la page (~150 caractères, utilisée par Google).">
  <meta name="author" content="Robin HOTTON">
  <meta name="robots" content="index, follow">
</head>
```

2. Open Graph (partage social)

```
<meta property="og:title" content="Titre pour Facebook/LinkedIn">
<meta property="og:description" content="Description pour partage social">
<meta property="og:image" content="https://site.fr/image-partage.jpg">
<meta property="og:url" content="https://site.fr/page">
```

♦ Tester son partage social

Utiliser **metatags.io** ou le Facebook Sharing Debugger pour prévisualiser comment votre page apparaîtra sur les réseaux sociaux avant publication.

E. Accessibilité de base

1. Alt-text sur les images

```
←!— ✓ Bon alt-text : décrit l'image pour un lecteur d'écran →  
  
  
←!— ✗ Alt-text inutile →  
  
  
←!— ✓ Image décorative : alt vide (pas absent) →  

```

2. ARIA et labels

```
←!— Bouton icône sans texte visible : il FAUT un label accessible →  
<button aria-label="Fermer la boîte de dialogue">  
  <span aria-hidden="true">×</span>  
</button>  
  
←!— Formulaire : associer label et input →  
<label for="email">Adresse e-mail</label>  
<input id="email" type="email" name="email" required>
```

♦ L'accessibilité n'est pas une option

Selon l'OMS, environ **15 % de la population mondiale** vit avec une forme de handicap. En France, depuis la loi RGAA, l'accessibilité est **obligatoire pour le secteur public** et fortement recommandée pour le privé.

3. Skip links

```
<body>
  <a href="#contenu-principal" class="skip-link">Aller au contenu principal</a>

  <header>...</header>
  <nav>...</nav>

  <main id="contenu-principal">
    ←!— contenu →
  </main>
</body>
```

Permet aux utilisateurs de lecteurs d'écran ou de clavier de sauter directement au contenu sans retraverser toute la navigation.

F. Conventions de nommage et organisation

1. Méthodologie BEM

BEM (*Block Element Modifier*) structure les noms de classes CSS pour éviter les collisions et rendre le HTML lisible :

```
<article class="card">
  <header class="card__header">
    <h2 class="card__title">Titre</h2>
  </header>
  <div class="card__body card__body--highlighted">
    <p class="card__text">Contenu</p>
  </div>
</article>
```

Pattern : `bloc__element--modificateur` .

Avantages :

- Pas de conflits de noms entre composants.
- Lisibilité immédiate (on voit que `card__title` appartient à `card`).
- Réutilisabilité facile.

2. Arborescence de fichiers recommandée

```
mon-site/
├─ index.html
├─ assets/
│  ├─ css/
│  │  └─ style.css
│  ├─ js/
│  │  └─ main.js
│  └─ images/
│     └─ fonts/
├─ components/
│  ├─ header.html
│  └─ footer.html
└─ pages/
   ├─ about.html
   └─ contact.html
```

3. Validation

Outils indispensables avant toute livraison :

- **W3C Markup Validation** — valide la syntaxe HTML.
- **WAVE** — audit accessibilité.
- **Lighthouse** (intégré à Chrome DevTools) — audit global (perf, ally, SEO, best practices).

Conclusion

1. À retenir

- `<!DOCTYPE html>` + `<meta charset>` + `<meta viewport>` = squelette minimal obligatoire.
- Utiliser les **balises sémantiques** (`<header>`, `<main>`, `<article>`, etc.) plutôt qu'empiler des `<div>`.
- **Hiérarchie de titres sans saut** : `h1` → `h2` → `h3`, jamais `h1` → `h3`.
- Sémantique > présentation : `<strong>` au lieu de `<b>`, `<em>` au lieu de `<i>`.
- Toutes les images doivent avoir un `alt` (descriptif ou vide pour les décoratives).
- **BEM** structure les classes CSS pour éviter les collisions.
- **Lighthouse** + **WAVE** + **W3C Validator** = les 3 outils à lancer avant livraison.

2. Pour aller plus loin

- **MDN Web Docs — HTML Element Reference**
- **BEM Methodology**
- **Web Content Accessibility Guidelines (WCAG) 2.1**
- **Front-end Checklist**
- **Référentiel Général d'Amélioration de l'Accessibilité (RGAA)**